K. J. Somaiya School of Engineering, Mumbai-77

(Somaiya Vidyavihar University)

Department of Computer Engineering

Web Development Laboratory Semester: III Academic Year: 2026-27

Course Name: Web Development Laboratory

(316U01L306) Semester: III

Date of Performance: 6/10/2026 DIV/ Batch No: A / 2

Student Name: Kehan Desai Roll No: 16010125050

Experiment No: 4

Title

Form Validation using jQuery for a Climate Intelligence and Heatwave Early Warning Web Portal

Aim of the Experiment

To design and implement client-side form validation using jQuery for a climate monitoring and heatwave alert registration form, ensuring that user-entered data such as name, mobile number, email, location, temperature, humidity, alert level, monitoring date, and notification preference is complete, valid, and meaningful before form submission.

Objectives of the Experiment

- To create an interactive HTML form for collecting user and climate-monitoring information.
- To use jQuery to validate text, numeric, email, date, and selection fields.
- To implement regular expressions for validating mobile numbers, email addresses, and location/station IDs.
- To use jQuery selectors, events, DOM manipulation, and class-based styling to display validation messages.
- To prevent form submission when invalid or incomplete data is entered.
- To apply form validation to a real-world climate intelligence and heatwave early-warning use case.

COs to be achieved

CO1: Developing webpages using HTML, CSS, JavaScript and jQuery.

Books / Journals / Websites references

1. jQuery API Documentation – Selectors, Events and DOM Manipulation.

2. MDN Web Docs – HTML Forms and JavaScript Form Validation.

3. W3Schools – jQuery Form Validation and Event Handling.

4. JavaScript and jQuery documentation/reference material prescribed by the course instructor.

Theory

Form validation is the process of checking whether the information entered by a user satisfies predefined rules before it is submitted or processed. Validation improves data quality, usability, and reliability of web applications.

jQuery is a JavaScript library that simplifies DOM selection, event handling, and manipulation. In form validation, jQuery selectors can identify form controls, while methods such as val(), trim(), addClass(), removeClass(), text(), and event handlers such as submit() and input() can be used to validate and provide immediate feedback.

Common validation rules include checking for empty fields, minimum and maximum length, valid email format, valid mobile number, acceptable numeric ranges, selected options, and valid dates. Regular expressions are useful for validating structured values such as mobile numbers, email addresses, and station identifiers.

In this experiment, jQuery validation is applied to a Climate Intelligence and Heatwave Early Warning Web Portal. The portal collects location and climate-observation information that can be used to display heatwave status and early-warning notifications.

Problem Statement

"jQuery Form Validation for Climate Monitoring and Heatwave Early Warning"

Develop a web-based climate monitoring registration form for the Climate Intelligence and Heatwave Early Warning Web Portal. The form should collect essential user and climate-observation information and use jQuery to validate the entered values before submission. Invalid or incomplete data must be identified and appropriate error messages must be displayed to the user.

Task 1: Create the Climate Monitoring Registration Form

Scenario

Design a registration and climate-observation form for users who will monitor local temperature conditions and receive heatwave early-warning notifications.

Requirements

- Create fields for Observer Name, Mobile Number, Email, Location/Area, Weather Station ID, Temperature (°C), Relative Humidity (%), Heatwave Alert Level, Monitoring Date, and Notification Preference.
- Use <form>, <label>, <input>, <select>, <option>, <fieldset> and <legend>.
- Use suitable input types such as text, email, number, date, and radio buttons.
- Provide Submit and Reset buttons.
- Assign unique id and name attributes to all fields.

Concepts Tested

- HTML Forms
- Form controls
- jQuery selectors
- DOM element identification

Deliverable: climate-alert.html

Task 2: Validate Required Text Fields

Scenario

Ensure that essential observer and location information is not left blank.

Requirements

- Check that Observer Name, Location/Area and Weather Station ID are not empty.
- Remove leading and trailing spaces using jQuery trim-related validation logic.
- Validate that Observer Name contains alphabetic characters and spaces only.
- Validate the Weather Station ID using a suitable regular expression.
- Display an error message beside each invalid field.

Concepts Tested

- jQuery selectors
- Conditions
- String validation
- DOM manipulation
- Required-field validation

Deliverable: climate-alert.html

Task 3: Validate Mobile Number, Email and Climate Values

Scenario

Verify that contact information and observed climate values follow the expected format and range.

Requirements

- Validate a 10-digit Indian mobile number using a suitable regular expression.
- Validate the email address using a suitable regular expression or HTML input validation.
- Validate Temperature (°C) within a meaningful input range, such as -10°C to 60°C.
- Validate Relative Humidity (%) within the range 0 to 100.
- Display specific error messages for invalid mobile number, email, temperature, or humidity.
- Do not allow the form to submit until all required values are valid.

Concepts Tested

- jQuery selectors
- Regular expressions
- Numeric validation
- String validation
- Form submit event

Deliverable: climate-alert.html

Output / Screenshot:

Students should execute climate-alert.html in a web browser and paste screenshots showing: (1) the climate monitoring registration form, (2) validation errors for invalid inputs, and (3) the success message after entering valid data.

Output screenshots are attached on pages 10-12.

Conclusion and Discussion:

The experiment demonstrates how jQuery can be used to validate user input on a web form before submission. Validation rules were applied to observer details, contact information, location and weather-station details, temperature, humidity, alert level, monitoring date, and notification preference.

Discussion: I tested the form with valid and invalid inputs. Empty fields, incorrect contact details and values outside the allowed ranges displayed error messages. Valid inputs displayed a success message, and Reset cleared the form. jQuery selectors, events, regular expressions, numeric comparisons and DOM manipulation helped me validate the data and provide immediate feedback.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Climate Monitoring Registration</title>
  <style>

    * { box-sizing: border-box; }
    body { margin: 0; background: #f1f7f3; color: #183c2b;
      font-family: Arial, sans-serif; line-height: 1.45; }
    main { max-width: 960px; margin: 24px auto; padding: 0 22px; }
    header { padding: 22px 26px; background: #216644;
      color: white; border-radius: 10px 10px 0 0; }
    h1 { font-size: 27px; margin: 0 0 6px; }
    header p { margin: 0; }
    form { padding: 22px 26px; background: white;
      border: 1px solid #cbded1; border-radius: 0 0 10px 10px; }
    .intro { margin: 0 0 18px; color: #516359; }
    fieldset { margin: 0 0 18px; padding: 16px;
      border: 1px solid #cbded1; border-radius: 6px; }
    legend { padding: 0 8px; font-weight: bold; }
    .grid { display: grid; grid-template-columns: 1fr 1fr;
      gap: 12px 22px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input:not([type="radio"]), select { width: 100%; padding: 10px;
      border: 1px solid #819b8c; border-radius: 4px;
      background: white; color: #172e20; font: inherit; }
    input:focus, select:focus, button:focus-visible {
      outline: 2px solid #216644; outline-offset: 2px; }
    .error { display: block; color: #b32323; font-size: 13px;
      min-height: 19px; margin-top: 4px; }
    .invalid { border: 2px solid #b32323 !important;
      background: #fff7f7 !important; }
    .choices { display: flex; flex-wrap: wrap; gap: 24px; }
    .choices label { font-weight: normal; margin: 0; }
    input[type="radio"] { accent-color: #216644; }
    button { padding: 11px 25px; border: 0; border-radius: 4px;
      background: #216644; color: white; font: inherit;
      font-weight: bold; cursor: pointer; margin-right: 8px; }
    button[type="reset"] { background: #e6efe9; color: #183c2b; }
    button:hover { filter: brightness(.94); }
    #success { margin: 18px 0 0; padding: 14px;
      background: #e7f6eb; border-left: 4px solid #287440; }
    #success h2 { margin: 0 0 6px; font-size: 19px; }
    #success p { margin: 5px 0; }
    footer { text-align: center; font-size: 13px;
      color: #526258; margin: 15px 0; }
    @media (max-width: 600px) {
      .grid { grid-template-columns: 1fr; }
      main { padding: 0 10px; margin: 12px auto; }
      form, header { padding: 18px; }
    }
  </style>
</head>
<body>
```

```
<main>
  <header>
    <h1>Climate Monitoring Registration</h1>
    <p>Climate Intelligence and Heatwave Early Warning Portal</p>
  </header>

  <form id="climateForm" novalidate>
    <p class="intro">Enter observer details and a climate observation.
      All fields are required.</p>
    <fieldset>
      <legend>Observer details</legend>
      <div class="grid">
        <div>
          <label for="observerName">Observer Name</label>
          <input id="observerName" name="observerName" type="text"
            placeholder="e.g. Kehan Desai" required
            aria-describedby="observerNameError">
          <span id="observerNameError" class="error"></span>
        </div>
        <div>
          <label for="mobile">Mobile Number</label>
          <input id="mobile" name="mobile" type="tel" inputmode="numeric"
            placeholder="10 digits, starting with 6 to 9" required
            aria-describedby="mobileError">
          <span id="mobileError" class="error"></span>
        </div>
        <div>
          <label for="email">Email</label>
          <input id="email" name="email" type="email" required
            placeholder="observer@example.com" aria-describedby="emailError">
          <span id="emailError" class="error"></span>
        </div>
        <div>
          <label for="location">Location / Area</label>
          <input id="location" name="location" type="text" required
            placeholder="e.g. Mumbai" aria-describedby="locationError">
          <span id="locationError" class="error"></span>
        </div>
      </div>
    </fieldset>
    <fieldset>
      <legend>Climate observation</legend>
      <div class="grid">
        <div>
          <label for="stationId">Weather Station ID</label>
          <input id="stationId" name="stationId" type="text" required
            placeholder="e.g. AWS001" aria-describedby="stationIdError">
          <span id="stationIdError" class="error"></span>
        </div>
        <div>
          <label for="monitoringDate">Monitoring Date</label>
          <input id="monitoringDate" name="monitoringDate" type="date"
            required aria-describedby="monitoringDateError">
          <span id="monitoringDateError" class="error"></span>
        </div>
        <div>
          <label for="temperature">Temperature (°C)</label>
          <input id="temperature" name="temperature" type="number"
            min="-10" max="60" step="any" placeholder="-10 to 60"
```

```
            required aria-describedby="temperatureError">
          <span id="temperatureError" class="error"></span>
        </div>
        <div>
          <label for="humidity">Relative Humidity (%)</label>
          <input id="humidity" name="humidity" type="number"
            min="0" max="100" step="any" placeholder="0 to 100"
            required aria-describedby="humidityError">
          <span id="humidityError" class="error"></span>
        </div>
        <div>
          <label for="alertLevel">Heatwave Alert Level</label>
          <select id="alertLevel" name="alertLevel" required
            aria-describedby="alertLevelError">
            <option value="">Select alert level</option>
            <option value="Normal">Normal</option>
            <option value="Watch">Watch</option>
            <option value="Warning">Warning</option>
            <option value="Severe">Severe</option>
          </select>
          <span id="alertLevelError" class="error"></span>
        </div>
      </div>
    </fieldset>
    <fieldset id="notificationGroup" aria-describedby="notificationError">
      <legend>Notification Preference</legend>

      <div class="choices">
        <label for="notifySms"><input id="notifySms" name="notification"
          type="radio" value="SMS" required> SMS</label>
        <label for="notifyEmail"><input id="notifyEmail" name="notification"
          type="radio" value="Email"> Email</label>
        <label for="notifyBoth"><input id="notifyBoth" name="notification"
          type="radio" value="Both"> Both</label>
      </div>
      <span id="notificationError" class="error"></span>
    </fieldset>
    <button type="submit">Submit</button>
    <button type="reset">Reset</button>
    <section id="success" role="status" hidden>
      <h2>Form submitted successfully!</h2>
      <p>All entered details passed validation.</p>
      <p id="summary"></p>
      <p>Demonstration only: no data is sent and no notifications are
        issued.</p>
    </section>
    <noscript>Please enable JavaScript to validate this form.</noscript>
  </form>
  <footer>Web Development Laboratory | Form validation using jQuery</footer>
</main>

<script src="jquery-3.7.1.min.js"></script>
<script>
$(function () {
  const fields = ["observerName", "mobile", "email", "location",
    "stationId", "temperature", "humidity", "alertLevel",
    "monitoringDate", "notification"];

  function showError(id, message) {
```

```
  const control = id === "notification"
    ? $("#notificationGroup") : $("#" + id);
  $("#" + id + "Error").text(message);
  control.removeClass("invalid").removeAttr("aria-invalid");
  if (message) control.addClass("invalid").attr("aria-invalid", "true");
  return message === "";
}

function validate(id) {
  const control = $("#" + id);
  const value = id === "notification"
    ? $("input[name='notification']:checked").val() || ""
    : control.val().trim();
  let error = "";
  if (value === "") {
    error = "This field is required.";
    if (id === "notification") error = "Select a notification preference.";
    if (id === "alertLevel") error = "Select a heatwave alert level.";
  } else {

    switch (id) {
      case "observerName":
        if (!/^[A-Za-z ]+$/.test(value))
          error = "Use alphabets and spaces only.";
        break;
      case "stationId":
        if (!/^AWS[0-9]{3}$/.test(value))
          error = "Use AWS followed by 3 digits, e.g. AWS001.";
        break;

      case "mobile":
        if (!/^[6-9][0-9]{9}$/.test(value))
          error = "Enter 10 digits starting with 6, 7, 8 or 9.";
        break;
      case "email":
        if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value) ||
            control[0].validity.typeMismatch)
          error = "Enter a valid email, e.g. observer@example.com.";
        break;
      case "temperature":
        if (!Number.isFinite(Number(value)) ||
            Number(value) < -10 || Number(value) > 60)
          error = "Temperature must be between -10 and 60 °C.";
        break;
      case "humidity":
        if (!Number.isFinite(Number(value)) ||
            Number(value) < 0 || Number(value) > 100)
          error = "Humidity must be between 0 and 100%.";
        break;
      case "monitoringDate": {
        const date = new Date(value + "T00:00:00");
        const today = new Date();
        today.setHours(0, 0, 0, 0);
        if (!/^\d{4}-\d{2}-\d{2}$/.test(value) ||
            Number.isNaN(date.getTime()) || date > today)
          error = "Select a valid date that is not in the future.";
        break;
      }
      case "alertLevel":
```

```
          if (!["Normal", "Watch", "Warning", "Severe"].includes(value))
            error = "Select a heatwave alert level.";
          break;
      }
    }
    return showError(id, error);
  }

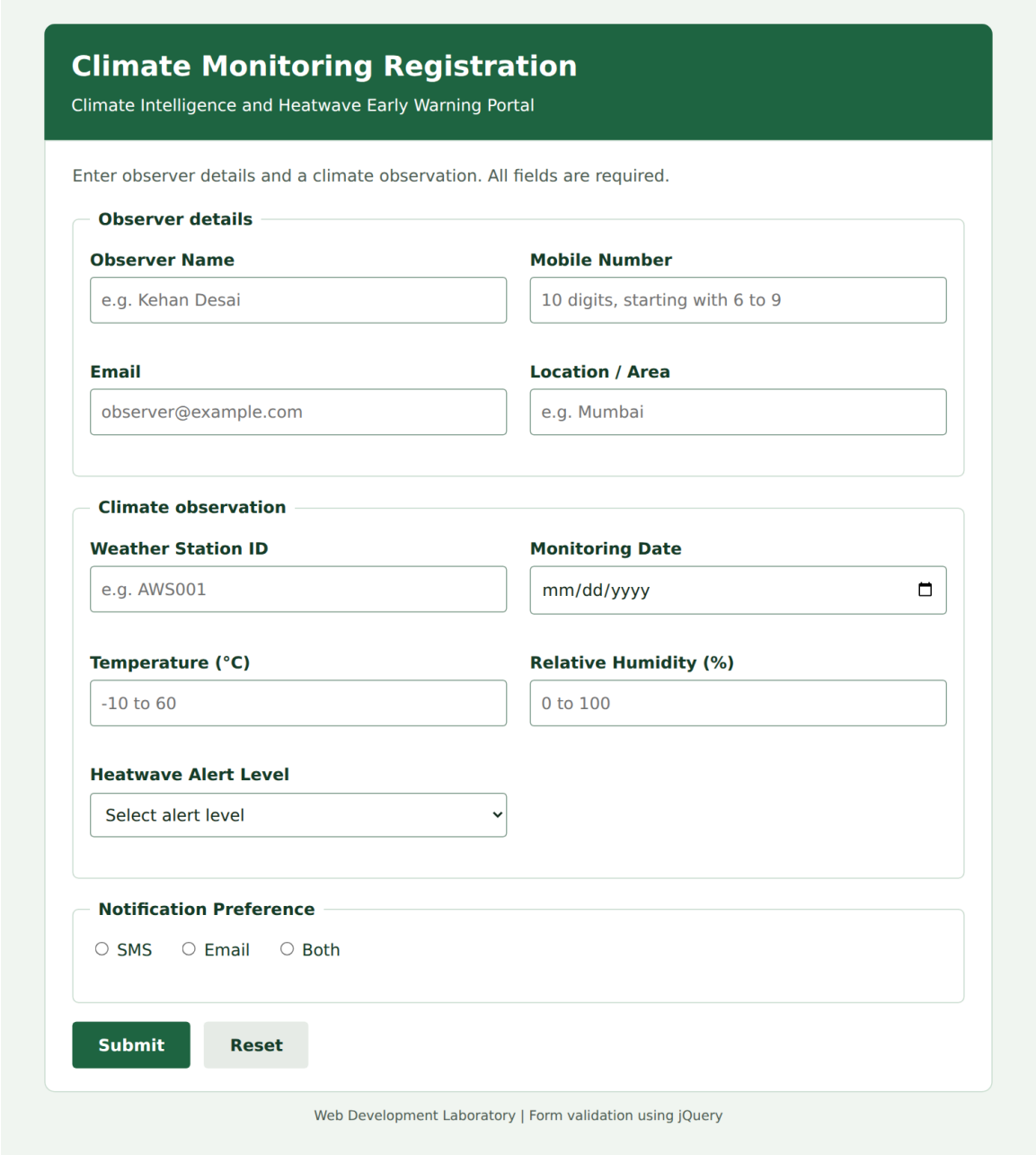
  $("#climateForm").on("submit", function (event) {
    event.preventDefault();
    $("#success").prop("hidden", true);
    $("input[type='text'], input[type='tel'], input[type='email']")
      .each(function () { $(this).val($(this).val().trim()); });
    let valid = true;
    fields.forEach(function (id) {
      if (!validate(id)) valid = false;
    });
    if (!valid) {
      $(".invalid").first().find("input").addBack("input, select")
        .first().trigger("focus");
      return;
    }
    const preference = $("input[name='notification']:checked").val();
    $("#summary").text($("#observerName").val() + " | " +
      $("#location").val() + " | " + $("#stationId").val() + " | " +
      $("#temperature").val() + " °C | " + $("#humidity").val() + "% | " +
      $("#alertLevel").val() + " | " + $("#monitoringDate").val() +
      " | Notifications: " + preference);
    $("#success").prop("hidden", false);
  });

  $("#climateForm input, #climateForm select").on("input change", function () {
    $("#success").prop("hidden", true);
    validate(this.name === "notification" ? "notification" : this.id);
  });

  $("#climateForm").on("reset", function () {
    $(".error").text("");
    $(".invalid").removeClass("invalid").removeAttr("aria-invalid");
    $("#success").prop("hidden", true);
    $("#summary").text("");
  });
});
</script>
</body>
</html>
```

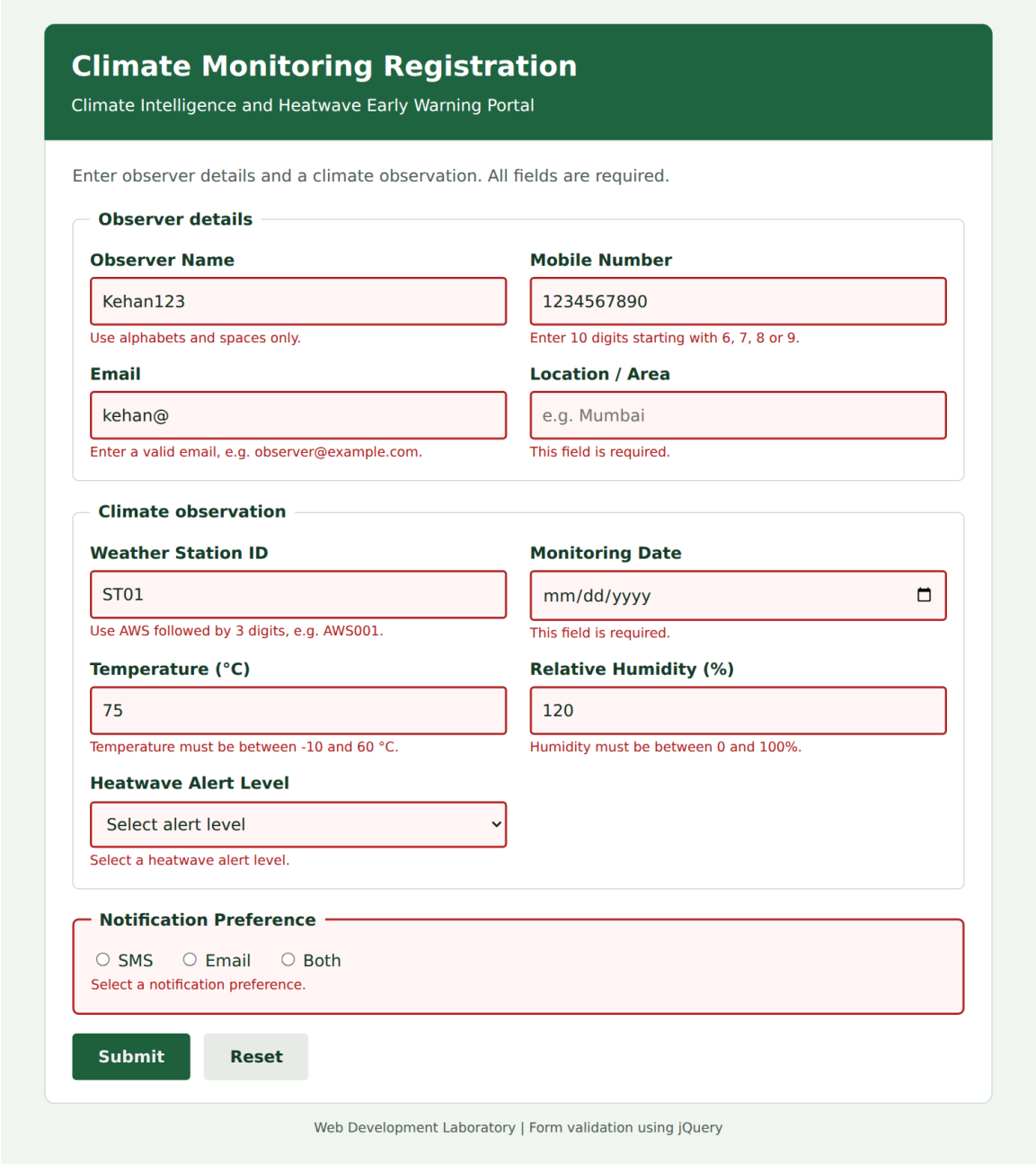
Output / Screenshot:

1. Climate monitoring registration form



Output / Screenshot:

2. Validation errors for invalid inputs



Output / Screenshot:

3. Success message after entering valid data

